Comparing ReLU, Tanh, Arctan, and Softsign: Optimization Dynamics and Initialization Scale in a Shallow MLP

Mohamed Elsabah Faris Elsabah

April 30, 2026

Abstract

Activation functions are a central design choice in neural networks, yet the rationale for selecting one activation over another is frequently reduced to convention rather than explicit analysis of optimization behavior. This project studies how activation functions influence gradient flow, training dynamics, and convergence in a controlled setting. We implement a multilayer perceptron (MLP) from scratch and conduct a controlled ablation study on a nonlinear XOR binary classification task. We compare ReLU, tanh, arctan, and softsign under identical architecture, learning rate, optimizer, dataset generation procedure, and training budget, while sweeping Xavier/Glorot initialization scale across 0.5, 1.0, and 2.0. Each setting is repeated across 10 random seeds, and we report mean $\pm$ standard deviation for train/test accuracy, loss, derivative-based diagnostics, pre-activation scale diagnostics, and activation-limit proxies.

The results show that activation comparisons are strongly initialization-scale dependent. At small initialization scale (0.5), ReLU learns reliably while bounded smooth activations are slower and more seed-sensitive, especially tanh and arctan. At moderate and larger scales (1.0 and 2.0), all four activations reach similar final test accuracy, with endpoint differences largely within run-to-run variation. However, derivative and pre-activation diagnostics reveal distinct optimization regimes: small-scale smooth activations remain closer to a local-linear regime for longer, while larger scales expose stronger saturation and derivative-tail differences. We conclude that activation choice should be analyzed jointly with initialization scale and training dynamics rather than endpoint accuracy alone.

Contents

1 Introduction **3**

2 Background **3**
2.1 Binary classification with neural networks . . . 3
2.2 Loss function (binary cross-entropy) . . . 4
2.3 Backpropagation and gradient flow . . . 4
2.4 Activation functions and their derivatives . . . 4

3 Experimental Design **5**
3.1 Research questions and hypotheses . . . 5
3.2 Controlled ablation logic . . . 5
3.3 Experiment settings (reproducibility table) . . . 6

4 Implementation **6**
4.1 Data generation (XOR) . . . 6
4.2 Model architecture . . . 7
4.3 Training algorithm . . . 7
4.4 Metrics . . . 7
4.5 Logging outputs and artifacts . . . 8

5 Results **9**
5.1 Multi-seed Xavier scale sweep . . . 9
5.2 Small-scale regime: delayed nonlinear learning . . . 9
5.3 Larger-scale regime: similar endpoints, different derivative tails . . . 11
5.4 Quantitative diagnostic summary . . . 12

6 Discussion **12**
6.1 Endpoint accuracy is not enough . . . 12
6.2 Small-scale smooth activations can remain near-linear . . . 13
6.3 Why ReLU is more reliable at small scale . . . 13
6.4 Larger scale reduces endpoint differences . . . 13
6.5 Derivative tails reveal hidden differences . . . 13

7 Limitations **13**

8 Future Work **14**

9 Conclusion **14**

10 Related Work and References **14**

A Activation Definitions and Derivatives **15**
A.1 ReLU . . . 15
A.2 tanh . . . 15
A.3 arctan . . . 15
A.4 softsign . . . 15

B Reproducibility Checklist **15**

C How to reproduce (commands) **16**

1 Introduction

Neural networks are trained via gradient-based optimization. While architecture, data pipelines, and optimizers receive major emphasis, a foundational design decision—the activation function—is often justified by convention (e.g., "use ReLU") rather than a careful analysis of why it improves optimization. In practice, ReLU-like activations dominate modern deep learning despite known concerns such as dead units and nondifferentiability at zero [3, 1].

This report focuses on *gradient flow*: how gradients propagate through layers during backpropagation and how that propagation affects training speed, stability, and final performance [4, 1]. We distinguish two frequently conflated ideas:

- **Representational capacity:** whether a model class *can* represent a solution for the task.
- **Optimization efficiency (trainability):** whether gradient-based training *finds* a good solution within a finite training budget.

A model can be expressive enough but still train poorly due to vanishing gradients, exploding gradients, saturation, poor scaling, or regimes in which nonlinearities behave almost linearly [1].

Why XOR? We choose XOR because it is a minimal nonlinear classification problem: it is not linearly separable, but it is solvable with a small multilayer perceptron (MLP) [4, 1]. This makes it well suited for isolating optimization effects without confounds from complex real-world datasets, such as data leakage, high-dimensional noise, feature engineering, or domain-specific preprocessing.

Contributions. This project provides:

1. A from-scratch NumPy MLP implementation including forward propagation, backpropagation, parameter updates, and CSV logging.
2. A controlled activation comparison of ReLU, tanh, arctan, and softsign on noisy XOR.
3. A multi-seed Xavier/Glorot initialization-scale sweep over scales 0.5, 1.0, and 2.0.
4. Diagnostic measurements beyond accuracy, including derivative statistics, pre-activation scale statistics, gradient norms, and activation-limit proxies.
5. A discussion that separates endpoint accuracy from training dynamics and shows that activation comparisons can change across initialization regimes.

2 Background

2.1 Binary classification with neural networks

Binary classification maps inputs $x \in \mathbb{R}^d$ to labels $y \in \{0, 1\}$. In an MLP, we compute a sequence of affine transformations followed by nonlinearities [1]. For the architecture used in this work (two hidden layers), forward propagation is:

$$h^{(1)} = \phi\Big(W^{(1)}x + b^{(1)}\Big), \tag{1}$$

$$h^{(2)} = \phi\Big(W^{(2)}h^{(1)} + b^{(2)}\Big), \tag{2}$$

$$y_{\text{pred}} = \sigma\Big(W^{(3)}h^{(2)} + b^{(3)}\Big), \tag{3}$$

where ϕ is the hidden activation function under study and $\sigma(\cdot)$ is the sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

We convert the probability prediction $y_{\text{pred}} \in (0, 1)$ to a class prediction using a threshold:

$$y_{\text{class}} = \begin{cases} 1, & y_{\text{pred}} \geq 0.5, \\ 0, & \text{otherwise.} \end{cases}$$

2.2 Loss function (binary cross-entropy)

We train by minimizing binary cross-entropy:

$$\mathcal{L}(y, y_{\text{pred}}) = -\big[y \log(y_{\text{pred}}) + (1 - y) \log(1 - y_{\text{pred}})\big].$$

This loss strongly penalizes confident wrong predictions and is standard for sigmoid-based binary classification [1].

2.3 Backpropagation and gradient flow

Backpropagation computes gradients of $\mathcal{L}$ with respect to parameters using the chain rule [4]. In deep networks, gradients are repeatedly multiplied by Jacobian terms. Two classic failure modes occur [1]:

- **Vanishing gradients:** gradient magnitudes shrink as they propagate backward, slowing learning in early layers.
- **Exploding gradients:** gradients grow, leading to unstable parameter updates.

Activation functions matter because their derivatives $\phi'(z)$ appear multiplicatively in backpropagation. If $\phi'(z)$ is frequently near zero, gradient propagation is suppressed [1].

2.4 Activation functions and their derivatives

We compare four activations in hidden layers:

- ReLU: $\phi(z) = \max(0, z)$, derivative $\phi'(z) = \begin{cases} 0 & z < 0 \\ 1 & z > 0 \end{cases}$.
- tanh: $\phi(z) = \tanh(z)$, derivative $\phi'(z) = 1 - \tanh^2(z)$.
- arctan: $\phi(z) = \arctan(z)$, derivative $\phi'(z) = \frac{1}{1+z^2}$.
- softsign: $\phi(z) = \frac{z}{1+|z|}$, derivative $\phi'(z) = \frac{1}{(1+|z|)^2}$.

Why these? ReLU is a widely used default in modern deep learning [3, 1]. tanh is a classical smooth bounded nonlinearity. arctan and softsign are smooth alternatives with different saturation profiles. Comparing them under identical conditions helps isolate how nonlinearity shape affects optimization dynamics.

Table 1: Qualitative properties of the studied activations (see [1, 3]).

Activation	**Range**	**Derivative behavior**	**Primary risk**
ReLU	$[0, \infty)$	0 for $z < 0$, 1 for $z > 0$	Dead or inactive units when $z \leq 0$ frequently
tanh	$(-1, 1)$	Peaks at 1 near 0, decays toward 0 as $\lvert z \rvert$ grows	Saturation $\Rightarrow$ vanishing gradients
arctan	$\left(-\frac{\pi}{2}, \frac{\pi}{2}\right)$	$\frac{1}{1+z^2}$ decays polynomially	Derivative shrinkage for large $\lvert z \rvert$
softsign	$(-1, 1)$	$\frac{1}{(1+\lvert z \rvert)^2}$ decays polynomially	Saturation-like regimes; slower updates

3 Experimental Design

3.1 Research questions and hypotheses

RQ1: How does activation choice affect convergence speed and final test accuracy under a fixed MLP architecture?
RQ2: How does Xavier/Glorot initialization scale interact with activation choice?
RQ3: Do smooth activations reach similar endpoint accuracy while exhibiting different derivative and pre-activation diagnostics?

These hypotheses are motivated by standard discussions of ReLU-style activations, saturation-induced gradient attenuation, and the importance of initialization scale in feedforward networks [3, 1, 2].
We hypothesize:

- **H1:** ReLU will be more reliable under small initialization scale because its piecewise-linear kink introduces nonlinearity even when pre-activations are small.
- **H2:** Bounded smooth activations will be more sensitive to initialization scale, especially when pre-activations remain close to zero and the network behaves nearly linearly.
- **H3:** At moderate and larger initialization scales, the four activations may reach similar final test accuracy, while derivative and pre-activation diagnostics still reveal different optimization regimes.

3.2 Controlled ablation logic

To make comparisons controlled, we hold the following constant within each activation/scale setting:

- dataset generation procedure (noisy XOR)
- architecture and parameter count
- optimizer and learning rate

- number of epochs
- train/test split rule
- evaluation metric and classification threshold

The studied factors are the hidden-layer activation function ϕ and the Xavier initialization scale. Each setting is repeated across 10 random seeds.

3.3 Experiment settings (reproducibility table)

Table 2: Experiment configuration used for the final Xavier scale sweep.

Item	**Value**
Task	XOR binary classification in $\mathbb{R}^2$
Architecture	$[2, 16, 16, 1]$ with sigmoid output
Hidden activations	ReLU, tanh, arctan, softsign
Loss	Binary cross-entropy [1]
Training epochs	3000
Optimizer	Full-batch gradient descent (no momentum)
Learning rate η	0.05
Dataset size n	300
Train/test split	75% train, 25% test
Noise / data generation	XOR in $\mathbb{R}^2$ (quadrant rule) with Gaussian input noise $\sigma = 0.1$; labels are assigned before adding noise.
Initialization	Xavier/Glorot normal [2], multiplied by scale $s \in \{0.5, 1.0, 2.0\}$: $W_{ij}^{(\ell)} \sim \mathcal{N}\left(0, \left[s\sqrt{\frac{2}{n_{\ell-1}+n_\ell}}\right]^2\right)$, $b = 0$.
Random seeds	$0, 1, \ldots, 9$
Decision threshold	0.5
Outputs logged	Train/test loss and accuracy, derivative diagnostics, pre-activation diagnostics, gradient norms/RMS, activation-limit proxy

4 Implementation

4.1 Data generation (XOR)

We generate an XOR-style binary classification dataset in $\mathbb{R}^2$ using a quadrant-based labeling rule. We first sample inputs

$$x_i = (x_{i1}, x_{i2}) \sim \mathrm{Unif}([-1, 1]^2), \quad i = 1, \ldots, n.$$

Labels are assigned by the XOR-of-signs condition:

$$y_i = \mathbf{1}\{x_{i1}x_{i2} < 0\},$$

so points in opposite-sign quadrants (II and IV) receive label 1, while points in same-sign quadrants (I and III) receive label 0.

To make the task slightly less trivial and to better expose optimization effects, we perturb inputs with isotropic Gaussian noise:

$$\tilde{x}_i = x_i + \epsilon_i, \qquad \epsilon_i \sim \mathcal{N}(0, \sigma^2 I_2),$$

with $\sigma = 0.1$ in our experiments. Labels are assigned before adding noise and then kept fixed.

4.2 Model architecture

We use a multilayer perceptron with two hidden layers of width 16 and a scalar sigmoid output:

$$[2, 16, 16, 1].$$

This architecture is intentionally small for interpretability while being expressive enough to solve XOR.

4.3 Training algorithm

At each epoch:

1. forward propagation computes train predictions,
2. train loss and train accuracy are computed,
3. backpropagation computes gradients $\nabla_\theta \mathcal{L}$,
4. derivative, pre-activation, and gradient diagnostics are logged,
5. test loss and test accuracy are computed using the same pre-update model state,
6. parameters update: $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$.

4.4 Metrics

Accuracy.

$$\hat{y}_{\text{class},i} = \mathbf{1}\{\hat{y}_{\text{pred},i} \geq 0.5\}, \qquad \text{Accuracy} = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}\{\hat{y}_{\text{class},i} = y_i\}.$$

Gradient norms and RMS. For each layer ℓ, we compute the Frobenius norm and RMS of the gradient of the weight matrix:

$$\|\nabla_{W^{(\ell)}} \mathcal{L}\|_F, \qquad \text{RMS}(\nabla_{W^{(\ell)}} \mathcal{L}) = \sqrt{\frac{1}{|\nabla_{W^{(\ell)}} \mathcal{L}|} \sum_j (\nabla_{W^{(\ell)}} \mathcal{L})_j^2}.$$

These values provide compact signals for gradient attenuation or unusually large updates.

Derivative diagnostics. For hidden-layer pre-activations z and hidden activation ϕ, we log statistics of $|\phi'(z)|$:

- $dphi_small_rate = \Pr(|\phi'(z)| < 10^{-3})$,
- $dphi_mean_abs = \mathbb{E}|\phi'(z)|$,
- $dphi_p10_abs$ and $dphi_p01_abs$, the 10th and 1st percentiles of $|\phi'(z)|$,
- $dphi_log_mean = \mathbb{E}[\log(|\phi'(z)| + 10^{-12})]$.

The percentile diagnostics are especially useful because low-tail derivatives identify parts of the network where gradient flow is weakest.

Pre-activation diagnostics. To diagnose whether a network is operating near the local-linear region of an activation, we log:

- $z_mean_abs = \mathbb{E}|z|$,
- z_p90_abs, the 90th percentile of $|z|$,
- $z_near0_rate = \Pr(|z| \leq 0.1)$.

Activation-limit proxy. Because different activations have different ranges and failure modes, we also log a qualitative activation-limit proxy:

- **ReLU:**
$$D_{\mathrm{ReLU}} = \Pr(z \leq 0),$$
which corresponds to zero ReLU output.
- **tanh:**
$$S_{\tanh} = \Pr(|\tanh(z)| > 0.95).$$
- **softsign:**
$$S_{\mathrm{softsign}} = \Pr\left(\left|\frac{z}{1+|z|}\right| > 0.95\right).$$
- **arctan:**
$$S_{\arctan} = \Pr(|\arctan(z)| > 0.95 \cdot (\pi/2)).$$

Metric comparability across activations. The activation-limit proxy is *not strictly comparable* across activations because each activation has a different range and failure mode. Therefore, it is treated as qualitative supporting evidence. The derivative and pre-activation diagnostics provide more directly comparable information about gradient flow and operating scale.

4.5 Logging outputs and artifacts

The repository stores per-seed CSV logs, per-setting summary CSVs with mean $\pm$ standard deviation over seeds, and a final summary table. The report figures are generated from the aggregated summary CSVs.

5 Results

5.1 Multi-seed Xavier scale sweep

Table 3 reports final test accuracy across 10 random seeds for each activation and Xavier initialization scale. Values are taken from the final epoch ($t = 2999$) of the aggregated CSV logs.

Table 3: Final test accuracy across 10 seeds for each activation and Xavier initialization scale.

Activation	**Scale 0.5**	**Scale 1.0**	**Scale 2.0**
ReLU	$\mathbf{0.9067 \pm 0.0260}$	0.9080 ± 0.0276	$\mathbf{0.9120 \pm 0.0247}$
tanh	0.7573 ± 0.1431	0.9093 ± 0.0278	0.9107 ± 0.0231
arctan	0.7000 ± 0.1503	$\mathbf{0.9107 \pm 0.0304}$	0.9107 ± 0.0239
softsign	0.8640 ± 0.0605	0.9053 ± 0.0227	0.9107 ± 0.0231

The main pattern is not that one activation universally dominates. Instead, the result is regime-dependent. At scale 0.5, ReLU is substantially more reliable than tanh and arctan, while softsign is intermediate. At scales 1.0 and 2.0, all four activations converge to similar final test accuracy, with differences largely within one standard deviation.

5.2 Small-scale regime: delayed nonlinear learning

Figure 1 shows test accuracy over training for Xavier scale 0.5. ReLU learns reliably, while tanh and arctan have slower and more seed-sensitive trajectories. This supports the interpretation that small initialization can keep bounded smooth activations close to their locally linear region for a long portion of training.

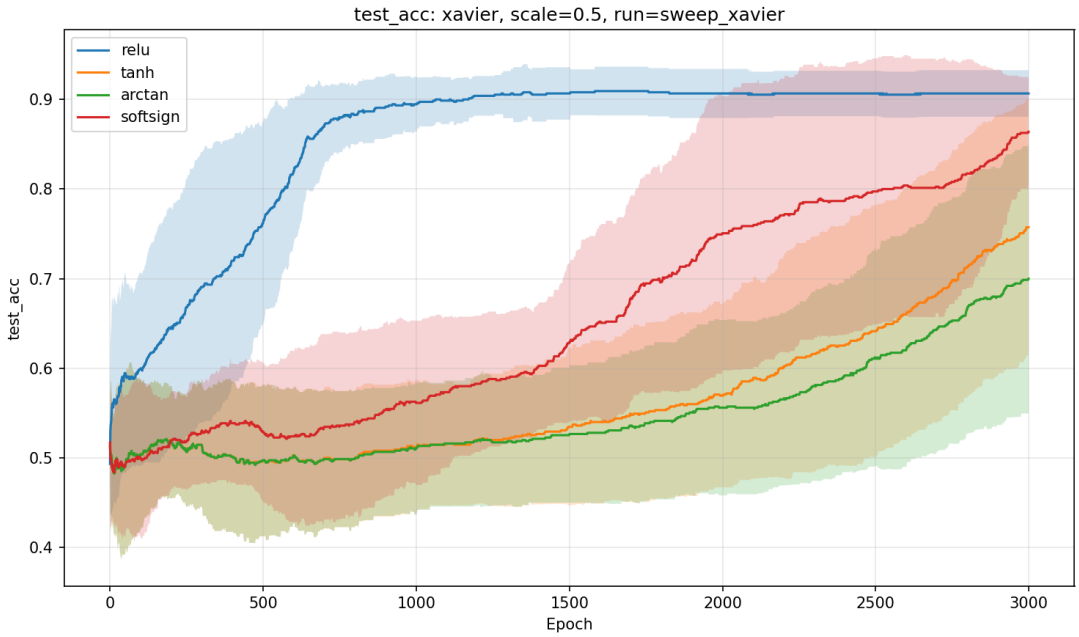

Figure 1: Test accuracy vs. epoch under Xavier initialization scale 0.5. Curves show mean $\pm$ standard deviation over 10 seeds.

Figure 2 supports this mechanism by plotting the fraction of hidden pre-activations close to zero. Larger z_near0_rate indicates that a greater fraction of units are operating near the local-linear region of the activation function. At Xavier scale 0.5, the smooth activations maintain a larger near-zero fraction than ReLU for much of training, which supports the interpretation that they spend longer in an approximately linear regime. This is especially relevant for XOR, where a nearly linear hidden representation is poorly matched to the nonlinear decision boundary.

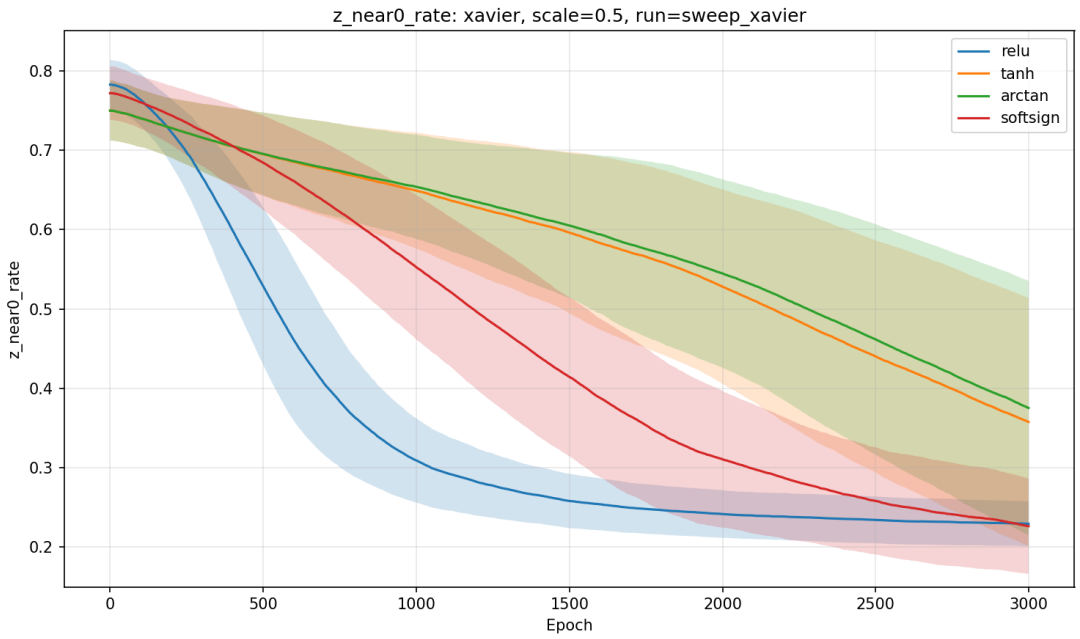

Figure 2: Fraction of hidden pre-activations satisfying $|z| \leq 0.1$ under Xavier scale 0.5. Larger values indicate that more hidden units are operating near the local-linear region.

Figure 3 shows the 90th percentile of $|z|$. As training progresses, larger z_p90_abs indicates that units are moving away from the local-linear region and into more nonlinear parts of the activation functions.

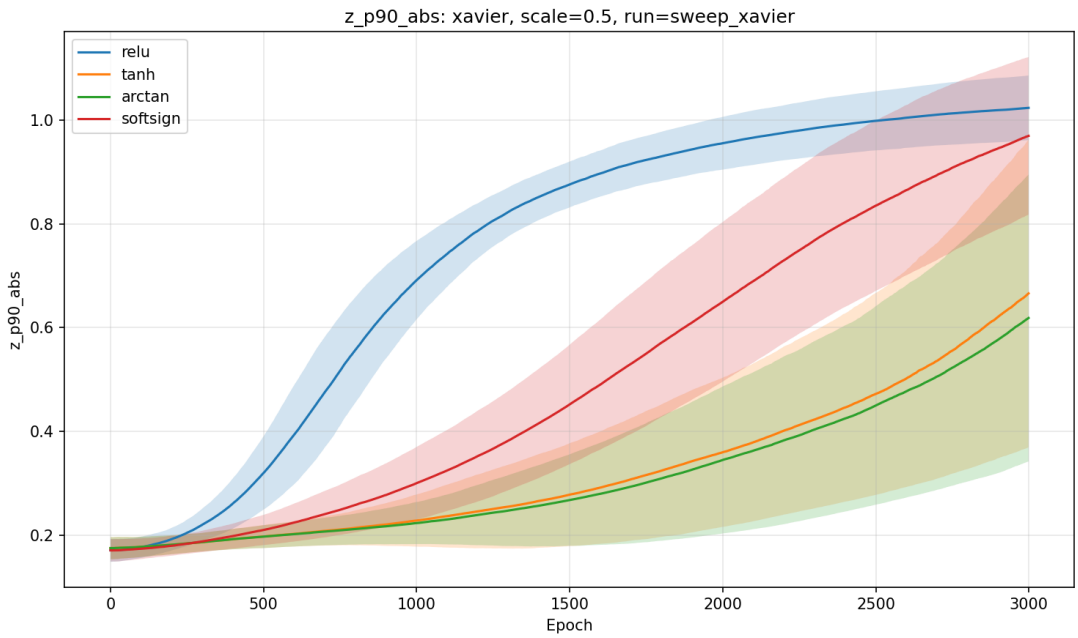

Figure 3: 90th percentile of hidden pre-activation magnitude under Xavier scale 0.5.

5.3 Larger-scale regime: similar endpoints, different derivative tails

Figure 4 shows that at Xavier scale 2.0, all four activations reach very similar final test accuracy. Endpoint performance alone would therefore suggest that activation choice matters little in this regime.

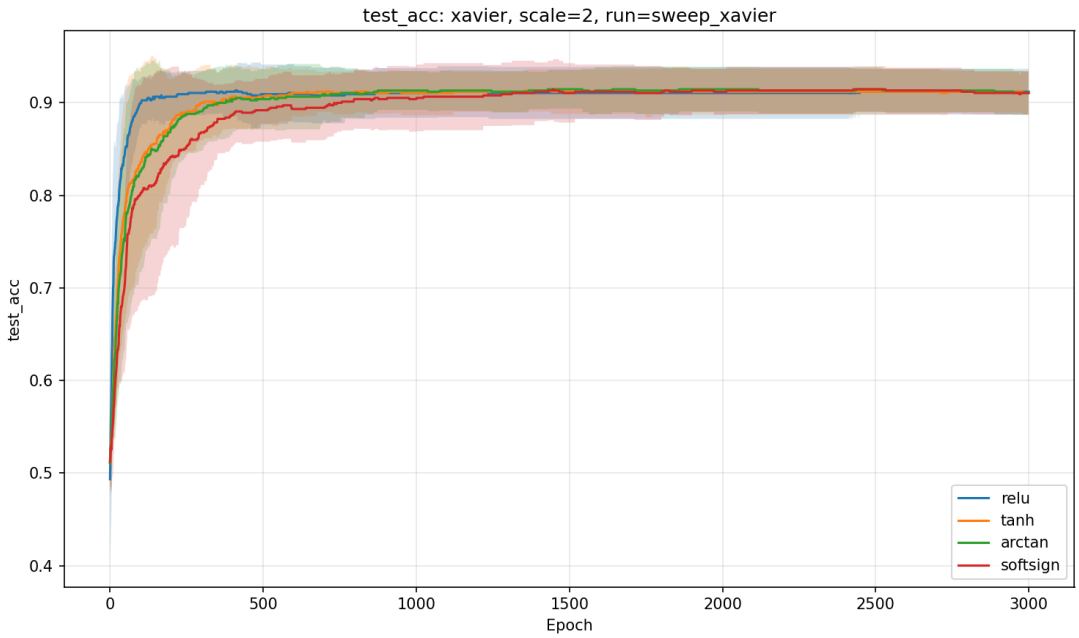

Figure 4: Test accuracy vs. epoch under Xavier initialization scale 2.0. Curves show mean $\pm$ standard deviation over 10 seeds.

However, Figure 5 shows that the smooth activations do not behave identically internally. The 1st percentile of $|\phi'(z)|$ captures the low-derivative tail. tanh develops a much smaller derivative tail than arctan and softsign, consistent with stronger saturation in a subset of units. Thus, similar final accuracy can mask different gradient-flow behavior.

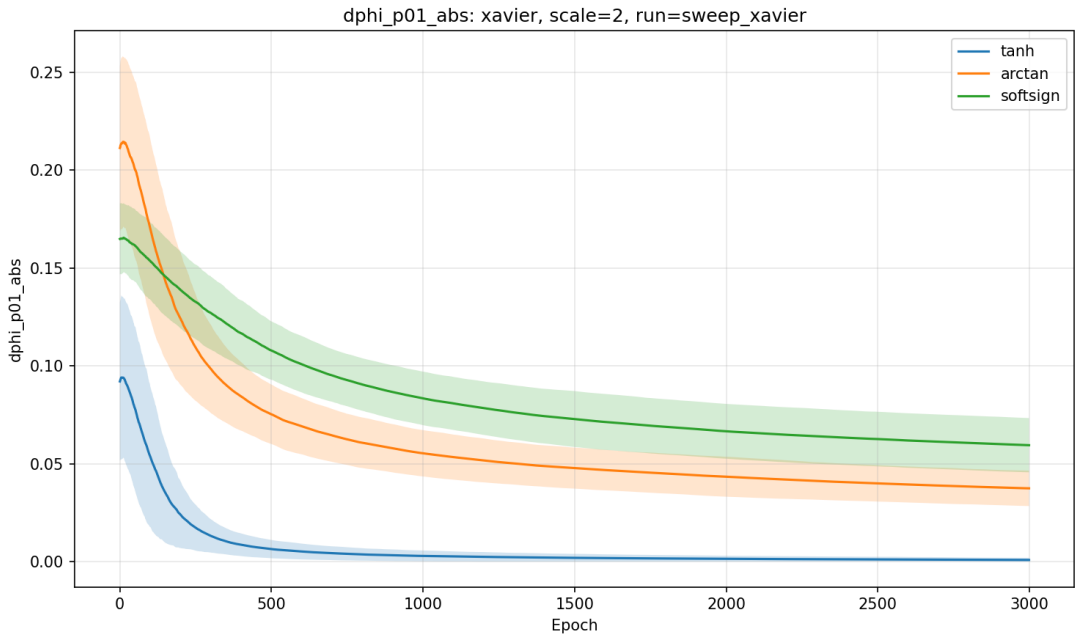

Figure 5: 1st percentile of hidden derivative magnitude for smooth activations under Xavier scale 2.0. Lower values indicate a weaker derivative tail and more severe local saturation.

5.4 Quantitative diagnostic summary

The final logs also show systematic changes in pre-activation scale. For ReLU, z_p90_abs increases from 1.024 at scale 0.5 to 1.800 at scale 2.0. For tanh, z_p90_abs increases from 0.666 to 2.382, while final z_near0_rate decreases from 0.358 at scale 0.5 to 0.072 at scale 2.0. This supports the interpretation that larger initialization scale moves the model away from the local-linear regime.

For smooth activations, the derivative tail also separates activations at scale 2.0: tanh has $dphi_p01_abs \approx 0.00086$, arctan has 0.03745, and softsign has 0.05949. These values indicate that tanh develops a much more extreme low-derivative tail, while arctan and softsign retain larger low-percentile derivatives despite similar final test accuracy.

6 Discussion

6.1 Endpoint accuracy is not enough

Final accuracy alone does not fully characterize training quality. At scales 1.0 and 2.0, all four activations reach similar final test accuracy. However, their learning curves, variance, pre-activation distributions, and derivative tails differ. This shows why activation comparisons should include training trajectories and diagnostic metrics, not only final endpoints.

6.2 Small-scale smooth activations can remain near-linear

At scale 0.5, tanh and arctan perform substantially worse and with higher variance than ReLU. The pre-activation diagnostics support a mechanism: many hidden pre-activations remain close to zero, where tanh and arctan are locally similar and approximately linear by their Taylor expansions [5]:

$$\tanh(z) = z - \frac{z^3}{3} + O(z^5), \qquad \arctan(z) = z - \frac{z^3}{3} + O(z^5).$$

Thus, when z remains small, these activations induce similar effective transformations and may not introduce enough useful nonlinearity early in training. This helps explain why a network with bounded smooth activations may be expressive in principle but slow or unstable to optimize in practice.

6.3 Why ReLU is more reliable at small scale

ReLU has a kink at zero and a derivative of one on the positive side. Even when initialization scale is small, ReLU can introduce a piecewise-linear partition of the input space and maintain strong gradient signal through active units. Although inactive ReLU units are common, the net effect in this experiment is more reliable optimization at small scale.

6.4 Larger scale reduces endpoint differences

At scales 1.0 and 2.0, smooth activations catch up in final test accuracy. This indicates that their poor small-scale performance is not a representational limitation on XOR. Instead, it is an optimization-scale effect: once pre-activations are sufficiently spread out, the model can exploit nonlinearity more effectively.

6.5 Derivative tails reveal hidden differences

At scale 2.0, endpoint accuracy is similar across activations, but *dphi_p01_abs* reveals meaningful internal differences. tanh develops the smallest derivative tail, consistent with stronger saturation for some units. arctan and softsign retain larger low-percentile derivatives. Therefore, similar accuracy does not imply identical gradient-flow behavior.

7 Limitations

- **Synthetic task.** The study uses a noisy XOR-style dataset. This is useful for controlled analysis but does not guarantee transfer to real-world datasets.
- **Single architecture.** Only one MLP architecture, $[2, 16, 16, 1]$, was tested. Activation behavior can change with depth, width, and parameter count.
- **Full-batch gradient descent only.** The optimizer is deliberately simple for interpretability. Results may change under SGD, momentum, Adam, learning-rate schedules, or regularization.
- **Fixed hyperparameters across activations.** A shared training recipe improves fairness, but it may disadvantage activations that benefit from different learning rates or training horizons.
- **Xavier scale sweep only.** The final analysis sweeps Xavier/Glorot scale, but does not fully analyze He initialization or other initialization schemes.

- **Diagnostic proxies.** Derivative and pre-activation diagnostics are informative but still indirect. They help explain optimization behavior, but they are not a complete theory of trainability.
- **Limited statistical testing.** We report mean $\pm$ standard deviation over 10 seeds. Formal hypothesis tests or confidence intervals could further quantify which differences are statistically reliable.

8 Future Work

- **He and Gaussian initialization sweeps.** Extend the same multi-seed analysis to He and fixed Gaussian initialization to compare initialization families, not only Xavier scale.
- **Time-to-threshold metrics.** Report the epoch at which each run first reaches target test accuracy levels such as 0.85 or 0.90. This would quantify convergence speed more directly.
- **Architecture sweep.** Evaluate deeper and wider MLPs to test whether the observed regime dependence changes with depth or width.
- **Optimizer and schedule sweep.** Evaluate SGD with momentum, Adam, and learning-rate schedules to test whether activation-specific behavior persists under stronger optimizers.
- **Hyperparameter fairness vs. optimality.** Tune learning rate and training horizon per activation under a consistent validation protocol to separate fixed-recipe fairness from activation-specific optimal performance.
- **Broader tasks.** Replicate the study on additional synthetic problems and at least one real dataset to assess whether the XOR findings transfer beyond this controlled setting.

9 Conclusion

Activation choice materially affects trainability, but the effect is strongly dependent on initialization scale. In the small-scale Xavier regime, ReLU is substantially more reliable than the bounded smooth activations, while tanh and arctan are slower and more seed-sensitive. In moderate and larger-scale regimes, all four activations reach similar final test accuracy, so there is no universal endpoint winner in this experiment.

The stronger conclusion is that activation comparisons based on one seed or one initialization scale can be misleading. ReLU, tanh, arctan, and softsign differ not only in final accuracy, but also in how they move through optimization regimes. Pre-activation scale diagnostics and derivative-tail metrics show that smooth activations can remain near-linear at small scale and can develop different saturation behavior at larger scale. Therefore, activation choice should be evaluated jointly with initialization scale, convergence dynamics, and gradient-flow diagnostics.

10 Related Work and References

References

[1] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, Cambridge, MA, 2016.

[2] X. Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 249–256, 2010.

[3] V. Nair and G. E. Hinton. Rectified linear units improve restricted Boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML)*, pages 807–814, 2010.

[4] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.

[5] T. M. Apostol. *Calculus, Volume 1.* Wiley, 2nd edition, 1967.

A Activation Definitions and Derivatives

A.1 ReLU

$$\phi(z) = \max(0, z), \qquad \phi'(z) = \begin{cases} 0, & z < 0, \\ 1, & z > 0. \end{cases}$$

A.2 tanh

$$\phi(z) = \tanh(z), \qquad \phi'(z) = 1 - \tanh^2(z).$$

A.3 arctan

$$\phi(z) = \arctan(z), \qquad \phi'(z) = \frac{1}{1 + z^2}.$$

A.4 softsign

$$\phi(z) = \frac{z}{1 + |z|}, \qquad \phi'(z) = \frac{1}{(1 + |z|)^2}.$$

B Reproducibility Checklist

- Dataset size fixed at $n = 300$ with Gaussian input noise $\sigma = 0.1$.
- Architecture fixed at $[2, 16, 16, 1]$.
- Full-batch gradient descent used with learning rate $\eta = 0.05$.
- Xavier/Glorot normal initialization used with scales 0.5, 1.0, and 2.0.
- Each activation/scale setting repeated across seeds 0 through 9.
- Train/test split fixed at 75%/25% per seed.
- Logged train/test loss and accuracy, derivative diagnostics, pre-activation diagnostics, activation-limit proxies, and gradient norms/RMS.
- Stored per-seed logs, aggregated summary CSVs, and final summary CSVs under `results/`.

- Generated figures from aggregated summary CSVs.
- Included a `requirements.txt` specifying third-party dependencies.

C How to reproduce (commands)

The finalized code, report figures, and full command list are available in the project repository:

`https://github.com/MBELSABAH/activation-gradient-flow`

```
# Clone the repository
git clone https://github.com/MBELSABAH/activation-gradient-flow.git
cd activation-gradient-flow

# The finalized project is on the main branch.
git checkout main

# Create and activate a virtual environment
python3 -m venv .venv
source .venv/bin/activate # macOS/Linux
# .venv\Scripts\activate # Windows (PowerShell)

# Install dependencies
pip install -r requirements.txt

# Smoke test
python3 src/train.py --run-tag smoke --seeds 0 --epochs 50 \
  --scales 1.0 --inits xavier --activations relu,tanh

# Final Xavier sweep used in this report
python3 src/train.py --run-tag sweep_xavier --inits xavier \
  --scales 0.5,1.0,2.0 \
  --seeds 0,1,2,3,4,5,6,7,8,9 \
  --epochs 3000 \
  --activations relu,tanh,arctan,softsign
```

Final figures are generated from the aggregated summary CSVs using `src/plot_results.py`. The exact figure-generation commands are listed in the repository README.